

Scenario 1: Bad Container Image Deployment

Objective

Trace an incomplete Kubernetes rollout from a Datadog signal to the Deployment image and Amazon ECR, then recover safely.

Trigger the Incident

Run this from the repository root against your own lab environment:

```
./scripts/chaos/bad-deploy.sh banking banking-backend banking-backend
```

The script changes the live Deployment image to a nonexistent tag. Wait for Datadog and Kubernetes to observe the failed rollout before troubleshooting. Do not run another scenario until this one has been recovered.

Situation

The banking application still serves users, but a new release has not completed. Do not run the trigger yourself unless you are facilitating the lab.

Symptoms

- The rollout does not complete.
- New pods do not start while existing pods may remain healthy.

What You Know

- The issue began immediately after a banking backend release.
- Datadog shows the `banking-backend` Deployment with unavailable replicas and new pods that are not Ready.
- Existing pods may still serve traffic.

Start With Datadog

1. Open **Dashboards > SRE Lab Scenario Signals** and set the time range to the last 15 minutes.
2. In **Unavailable Replicas**, filter `kube_namespace:banking` and `kube_deployment:banking-backend`. Look for `kubernetes_state.deployment.replicas_unavailable` rising above zero.

3. Compare **Available Replicas**. Existing capacity may stay healthy while the new rollout is unavailable.
4. Open **Monitors > Manage Monitors > [SRE Lab] Deployment has unavailable replicas**. Confirm the alert group names `banking/banking-backend`.
5. Open **Infrastructure > Kubernetes > Deployments**, filter `kube_namespace:banking` `kube_deployment:banking-backend`, and inspect the Deployment and newest ReplicaSet.
6. Open **Events > Explorer**, use the same namespace/workload tags, and look for `Failed`, `ErrImagePull`, `ImagePullBackOff`, or image-pull messages.

Record the first signal timestamp, unavailable replica count, affected ReplicaSet, and event message before using `kubectl`.

Troubleshooting

First find the incomplete rollout and its new pods.

```
kubectl get deployments -A
kubectl get pods -A
kubectl rollout status deployment/banking-backend -n banking
```

Then inspect the newest pod, its events, and the image requested by the Deployment.

```
kubectl describe pod <pod-name> -n banking
kubectl get events -n banking --sort-by=.lastTimestamp
kubectl get deployment banking-backend -n banking -o yaml
```

Compare the configured tag with ECR.

```
aws ecr describe-images --repository-name sre-lab/banking-backend --region us-east-1
```

Important Clues

Connect the Datadog replica mismatch with `ErrImagePull` or `ImagePullBackOff`, Kubernetes events, and the ECR image list.

Root Cause

Record your conclusion from Datadog, Kubernetes, and ECR evidence. Confirm it with the instructor after the exercise.

Fix

Roll back to the last working revision.

```
kubectl rollout history deployment/banking-backend -n banking  
kubectl rollout undo deployment/banking-backend -n banking
```

Validation

```
kubectl rollout status deployment/banking-backend -n banking  
kubectl get pods -n banking
```

Confirm the application still works, unavailable replicas return to zero, and the Datadog monitor recovers.

DevOps Lesson

Validate an ECR tag before changing a Kubernetes Deployment. A healthy old ReplicaSet does not mean a release succeeded.

STAR Method

Situation

Summarize the failed rollout without naming the cause first.

Task

Explain the safe recovery objective.

Action

Describe the evidence path from Datadog to Kubernetes and ECR.

Result

State how rollout and observability recovery were verified.

Natural Spoken Version

Use the matching example in [DevOps STAR Scenarios](#) after completing the lab.